

Below are **4 lab questions with complete C programs**, covering the topics in your syllabus: singly linked list operations, linked stack, linked queue, and header nodes.

Lab 1: Singly Linked List Operations

Question

Write a menu-driven C program to implement a Singly Linked List and perform the following operations:

1. Create a list
2. Traverse/display the list
3. Search an element
4. Insert at beginning
5. Insert at end
6. Insert after a given element
7. Delete from beginning
8. Delete from end
9. Delete a given element
10. Count the number of nodes

C Program

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *head = NULL;

/* Insert at beginning */
void insertBeginning(int value)
{
    struct Node *newNode;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
    }
}
```

```

        return;
    }

    newNode->data = value;
    newNode->next = head;
    head = newNode;

    printf("%d inserted at beginning.\n", value);
}

/* Insert at end */
void insertEnd(int value)
{
    struct Node *newNode, *temp;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        temp = head;

        while (temp->next != NULL)
            temp = temp->next;

        temp->next = newNode;
    }

    printf("%d inserted at end.\n", value);
}

/* Insert after a given element */
void insertAfter(int key, int value)
{
    struct Node *temp = head;
    struct Node *newNode;

    while (temp != NULL && temp->data != key)
        temp = temp->next;

```

```

    if (temp == NULL)
    {
        printf("%d not found.\n", key);
        return;
    }

    newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;
    newNode->next = temp->next;
    temp->next = newNode;

    printf("%d inserted after %d.\n", value, key);
}

/* Display / Traverse */
void display()
{
    struct Node *temp = head;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    printf("Linked List: ");

    while (temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }

    printf("NULL\n");
}

/* Search */
void search(int value)
{
    struct Node *temp = head;
    int position = 1;

    while (temp != NULL)
    {
        if (temp->data == value)

```

```

        {
            printf("%d found at position %d.\n",
                value, position);
            return;
        }

        temp = temp->next;
        position++;
    }

    printf("%d not found.\n", value);
}

/* Delete from beginning */
void deleteBeginning()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty. Deletion not possible.\n");
        return;
    }

    temp = head;
    head = head->next;

    printf("%d deleted from beginning.\n", temp->data);

    free(temp);
}

/* Delete from end */
void deleteEnd()
{
    struct Node *temp, *prev;

    if (head == NULL)
    {
        printf("List is empty. Deletion not possible.\n");
        return;
    }

    if (head->next == NULL)
    {
        printf("%d deleted from end.\n", head->data);
        free(head);
        head = NULL;
        return;
    }

    temp = head;

```

```

while (temp->next != NULL)
{
    prev = temp;
    temp = temp->next;
}

prev->next = NULL;

printf("%d deleted from end.\n", temp->data);

free(temp);
}

/* Delete a given element */
void deleteValue(int value)
{
    struct Node *temp = head;
    struct Node *prev = NULL;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    if (head->data == value)
    {
        head = head->next;
        free(temp);

        printf("%d deleted.\n", value);
        return;
    }

    while (temp != NULL && temp->data != value)
    {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL)
    {
        printf("%d not found.\n", value);
        return;
    }

    prev->next = temp->next;
    free(temp);

    printf("%d deleted.\n", value);
}

```

```

/* Count nodes */
void countNodes()
{
    struct Node *temp = head;
    int count = 0;

    while (temp != NULL)
    {
        count++;
        temp = temp->next;
    }

    printf("Number of nodes = %d\n", count);
}

/* Main function */
int main()
{
    int choice, value, key;

    while (1)
    {
        printf("\n===== SINGLY LINKED LIST =====\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert After a Given Element\n");
        printf("4. Display / Traverse\n");
        printf("5. Search\n");
        printf("6. Delete from Beginning\n");
        printf("7. Delete from End\n");
        printf("8. Delete a Given Element\n");
        printf("9. Count Nodes\n");
        printf("10. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                insertBeginning(value);
                break;

            case 2:
                printf("Enter value: ");
                scanf("%d", &value);
                insertEnd(value);
                break;

```

```

    case 3:
        printf("Enter element after which to insert: ");
        scanf("%d", &key);

        printf("Enter value to insert: ");
        scanf("%d", &value);

        insertAfter(key, value);
        break;

    case 4:
        display();
        break;

    case 5:
        printf("Enter value to search: ");
        scanf("%d", &value);
        search(value);
        break;

    case 6:
        deleteBeginning();
        break;

    case 7:
        deleteEnd();
        break;

    case 8:
        printf("Enter value to delete: ");
        scanf("%d", &value);
        deleteValue(value);
        break;

    case 9:
        countNodes();
        break;

    case 10:
        printf("Program terminated.\n");
        exit(0);

    default:
        printf("Invalid choice.\n");
}
}

```

Lab 2: Stack Using Linked List

Question

Write a C program to implement a Stack using a Singly Linked List. Perform PUSH, POP, PEEK and DISPLAY operations. Demonstrate Stack Underflow and Overflow due to memory allocation failure.

C Program

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *top = NULL;

/* PUSH */
void push(int value)
{
    struct Node *newNode;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Stack Overflow.\n");
        return;
    }

    newNode->data = value;
    newNode->next = top;
    top = newNode;

    printf("%d pushed into stack.\n", value);
}

/* POP */
void pop()
{
    struct Node *temp;

    if (top == NULL)
    {
        printf("Stack Underflow.\n");
        return;
    }
}
```

```

    temp = top;

    printf("%d popped from stack.\n", temp->data);

    top = top->next;

    free(temp);
}

/* PEEK */
void peek()
{
    if (top == NULL)
    {
        printf("Stack is empty.\n");
        return;
    }

    printf("Top element = %d\n", top->data);
}

/* DISPLAY */
void display()
{
    struct Node *temp = top;

    if (top == NULL)
    {
        printf("Stack is empty.\n");
        return;
    }

    printf("Stack: ");

    while (temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }

    printf("NULL\n");
}

int main()
{
    int choice, value;

    while (1)
    {
        printf("\n===== LINKED STACK =====\n");
        printf("1. PUSH\n");

```

```

printf("2. POP\n");
printf("3. PEEK\n");
printf("4. DISPLAY\n");
printf("5. EXIT\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
case 1:
    printf("Enter value: ");
    scanf("%d", &value);
    push(value);
    break;

case 2:
    pop();
    break;

case 3:
    peek();
    break;

case 4:
    display();
    break;

case 5:
    exit(0);

default:
    printf("Invalid choice.\n");
}
}

return 0;
}

```

Example

PUSH 10
PUSH 20
PUSH 30

TOP
↓
30
↓
20
↓

10
↓
NULL

After POP :

TOP
↓
20
↓
10
↓
NULL

Lab 3: Queue Using Linked List

Question

Write a C program to implement a Queue using a Singly Linked List. Use FRONT and REAR pointers and perform ENQUEUE, DEQUEUE, PEEK and DISPLAY operations.

C Program

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

/* ENQUEUE */
void enqueue(int value)
{
    struct Node *newNode;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Queue Overflow.\n");
        return;
    }
}
```

```

    }

    newNode->data = value;
    newNode->next = NULL;

    if (rear == NULL)
    {
        front = rear = newNode;
    }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }

    printf("%d inserted into queue.\n", value);
}

/* DEQUEUE */
void dequeue()
{
    struct Node *temp;

    if (front == NULL)
    {
        printf("Queue Underflow.\n");
        return;
    }

    temp = front;

    printf("%d deleted from queue.\n", temp->data);

    front = front->next;

    if (front == NULL)
        rear = NULL;

    free(temp);
}

/* PEEK */
void peek()
{
    if (front == NULL)
    {
        printf("Queue is empty.\n");
        return;
    }

    printf("Front element = %d\n", front->data);
}

```

```

/* DISPLAY */
void display()
{
    struct Node *temp = front;

    if (front == NULL)
    {
        printf("Queue is empty.\n");
        return;
    }

    printf("Queue: ");

    while (temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }

    printf("NULL\n");
}

int main()
{
    int choice, value;

    while (1)
    {
        printf("\n===== LINKED QUEUE =====\n");
        printf("1. ENQUEUE\n");
        printf("2. DEQUEUE\n");
        printf("3. PEEK\n");
        printf("4. DISPLAY\n");
        printf("5. EXIT\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
                dequeue();
                break;

            case 3:

```

```

        peek();
        break;

    case 4:
        display();
        break;

    case 5:
        exit(0);

    default:
        printf("Invalid choice.\n");
    }
}

return 0;

```

Example

FRONT	REAR
↓	↓
10 → 20 → 30 → NULL	

After `ENQUEUE (40)` :

FRONT	REAR
↓	↓
10 → 20 → 30 → 40 → NULL	

After `DEQUEUE ()` :

FRONT	REAR
↓	↓
20 → 30 → 40 → NULL	

Lab 4: Singly Linked List with Header Node

Question

Write a C program to implement a Singly Linked List using a Header Node. The header node should maintain the number of data nodes. Perform insertion, deletion, searching and traversal operations.

C Program

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

/* Create header node */
struct Node *createHeader()
{
    struct Node *header;

    header = (struct Node *)malloc(sizeof(struct Node));

    if (header == NULL)
    {
        printf("Memory allocation failed.\n");
        exit(1);
    }

    header->data = 0;          /* Number of nodes */
    header->next = NULL;

    return header;
}

/* Insert at beginning */
void insertBeginning(struct Node *header, int value)
{
    struct Node *newNode;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;
    newNode->next = header->next;

    header->next = newNode;

    header->data++;

    printf("%d inserted.\n", value);
}
```

```

/* Insert at end */
void insertEnd(struct Node *header, int value)
{
    struct Node *newNode;
    struct Node *temp;

    newNode = (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed.\n");
        return;
    }

    newNode->data = value;
    newNode->next = NULL;

    temp = header;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;

    header->data++;

    printf("%d inserted.\n", value);
}

/* Display */
void display(struct Node *header)
{
    struct Node *temp = header->next;

    if (temp == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    printf("List: ");

    while (temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }

    printf("NULL\n");

    printf("Number of data nodes = %d\n", header->data);
}

```

```

}

/* Search */
void search(struct Node *header, int value)
{
    struct Node *temp = header->next;
    int position = 1;

    while (temp != NULL)
    {
        if (temp->data == value)
        {
            printf("%d found at position %d.\n",
                value, position);
            return;
        }

        temp = temp->next;
        position++;
    }

    printf("%d not found.\n", value);
}

/* Delete a value */
void deleteValue(struct Node *header, int value)
{
    struct Node *prev = header;
    struct Node *temp = header->next;

    while (temp != NULL && temp->data != value)
    {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL)
    {
        printf("%d not found.\n", value);
        return;
    }

    prev->next = temp->next;

    free(temp);

    header->data--;

    printf("%d deleted.\n", value);
}

int main()

```

```

{
    struct Node *header;
    int choice, value;

    header = createHeader();

    while (1)
    {
        printf("\n===== HEADER NODE LINKED LIST =====\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Display\n");
        printf("4. Search\n");
        printf("5. Delete a Given Element\n");
        printf("6. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                insertBeginning(header, value);
                break;

            case 2:
                printf("Enter value: ");
                scanf("%d", &value);
                insertEnd(header, value);
                break;

            case 3:
                display(header);
                break;

            case 4:
                printf("Enter value to search: ");
                scanf("%d", &value);
                search(header, value);
                break;

            case 5:
                printf("Enter value to delete: ");
                scanf("%d", &value);
                deleteValue(header, value);
                break;

            case 6:
                free(header);
                exit(0);
        }
    }
}

```

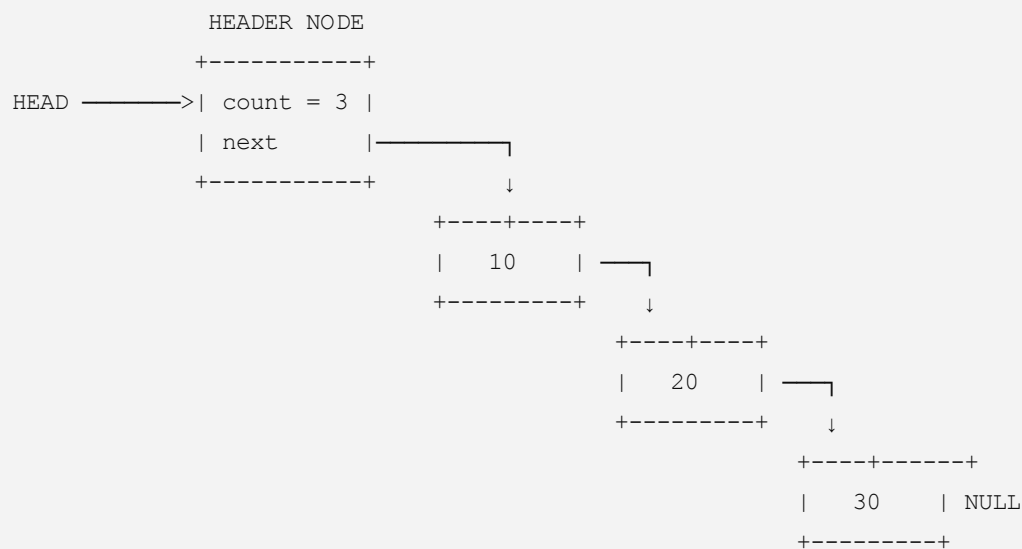
```

        default:
            printf("Invalid choice.\n");
        }
    }

    return 0;

```

Header Node Representation



Here, the header node stores:

```
header->data = 3
```

which represents the **number of actual data nodes**.

Lab Submission

For each program, students should write:

1. Aim
2. Problem Statement
3. Algorithm
4. C Program
5. Sample Input
6. Sample Output
7. Result

Suggested sequence for the lab

Experiment 1: Singly Linked List — Traversal, Searching, Insertion and Deletion

Experiment 2: Stack using Linked List — PUSH, POP, PEEK

Experiment 3: Queue using Linked List — ENQUEUE, DEQUEUE, PEEK

Experiment 4: Header Node — Linked List with Node Count

[See another version](#)